

High performance computing systems

Lab 6

Dept. of Computer Architecture
Faculty of ETI
Gdansk University of Technology

Robert Kałaska

The following example presents a basic implementation of generating and calculating sum of arithmetic sequence.

The code is divided into part which is executed on host (CPU of computer) and on device (GPU). The common difference is set to 1 and the first element of sequence is equal to 0. Each further elements of sequence are calculated as indexes of threads of CUDA application and are stored in common device memory. After generating all elements CPU calculates its sum.

Compilation instructions:

```
nvcc sample.cu -o sample
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <cuda_runtime.h>
```

```
__host__
```

```
void errexit(const char *s) {
```

```
    printf("\n%s",s);
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
//generate elements of sequence – parallel part
```

```
__global__
```

```
void calculate(int *result) {
```

```
int my_index=blockIdx.x*blockDim.x+threadIdx.x;

result[my_index]=my_index;

}
```

```
int main(int argc,char **argv) {

    long long result;

    int threadsinblock=1024;

    int blocksingrid=10000;


    int size = threadsinblock*blocksingrid;

    //memory allocation on host

    int *hresults=(int*)malloc(size*sizeof(int));

    if (!hresults) errorexit("Error allocating memory on the host");


    int *dresults=NULL;

    //memory allocation on device (GPU)

    if (cudaSuccess!=cudaMalloc((void **)&dresults,size*sizeof(int)))

        errorexit("Error allocating memory on the GPU");


    //call kernel on GPU – calculation are executed on GPU

    calculate<<<blocksingrid,threadsinblock>>>(dresults);

    if (cudaSuccess!=cudaGetLastError())

        errorexit("Error during kernel launch");

}
```

```

//copy all elements from device to host

if (cudaSuccess!=
=cudaMemcpy(hresults,dresults,size*sizeof(int),cudaMemcpyDeviceToHost))

    errexit("Error copying results");


//calculate sum of all elements on CPU side

result=0;


for(int i=0;i<size;i++) {

    result = result + hresults[i];

}


printf("\nThe final result is %lld\n",result);


//free resources

free(hresults);

if (cudaSuccess!=cudaFree(dresults))

    errexit("Error when deallocating space on the GPU");

}

```